

- This lab will review basic python concepts, classes, and memory map images.
 - You may want to refer to the text and your lecture notes during the lab as you solve the problems.
 - When approaching the problems, think before you code. Doing so is good practice and can help you lay out possible solutions.
 - Think of any possible test cases that can potentially cause your solution to fail!
 - **You must stay for the duration of the lab.** If you finish early, you may help other students. If you don't finish by the end of the lab, we recommend you complete it on your own time. Ideally, you should not spend more time than suggested for each problem.
 - Your TAs are available to answer questions in the lab, during office hours, and on Ed.
-

Vitamins (60 minutes)

1. Use python's list comprehension syntax to generate the following lists: (10 minutes)

a. `[1, -2, 4, -8, 16, -32, 64, -128]`

```
mylist = [(-2)**i for i in range(8)]
```

b. `[1, 11, 111, 1111, 11111, 111111, 1111111, 11111111]`

```
mylist = ["1"*i for i in range(1,8)]
```

2. Finish the python's list comprehension syntax. The result is a list of characters of the input repeated twice. **Do not use any arithmetic operators or additional libraries.**

Your answer must use `my_str` and `length`. (10 minutes)

```

    my_str[i] for j in range(2) for i in range(len(my_str))
print([_____])

my_str = "Python"
→ ["P", "y", "t", "h", "o", "n", "P", "y", "t", "h", "o", "n"]

my_str = "Java"
→ ["J", "a", "v", "a", "J", "a", "v", "a"]

```

3. Draw the memory diagram after each numbered line executes (5 diagrams for each q) for the following questions. (20 minutes):

(a)

```
lst = [1, [2, "abc"], [3, [4]], 7]      #1
lst_deepcopy = copy.deepcopy(lst)      #2
lst[0] = 10                             #3
lst[1][1] = "ABC"                       #4
lst_deepcopy[2][1][0] = 40              #5
```

(b)

```
lst1 = [[1, 2], [3, 4]]                # 1
lst2 = lst1[:]                          # 2
lst1[0][1] = 20                         # 3
lst2[1] = [30, 40]                      # 4
lst2[0].append(50)                      # 5
```

(c)

```
lst1 = [1, 2]                           # 1
lst2 = [lst1] * 3                        # 2
lst2[0][0] = 10                         # 3
lst2[1] = [20, 30]                      # 4
lst1.append(40)                         # 5
```

4. For each section below, **draw the memory image** for the execution of these lines of code. That is, you should draw the variables as they are organized in the call stack, and the data they each point to. Also write the outputs for (b) and (c) (20 minutes):

(a)

```
lst1 = [[1, 2], [3, 4]]
lst2 = lst1[:]
lst3 = copy.deepcopy(lst1)
lst1[0].append(5)
lst2[1][0] = 30
lst3[0][0] = 10
```

(b)

```
lst1 = [1, [2, 3]]
lst2=(lst1[:])
lst_copy= copy.deepcopy(lst2[1])
lst1[0] = 9
lst1[1].append(4)
lst_copy[1] = "a"
```

#Write the output in the space given for the following lines

```
print(lst1)
```

```
print(lst2)
```

```
print(lst_copy)
```

(c)

```
class Shelf:
    def __init__(self, items):
        self.items = items
    def add(self, x):
        self.items.append(x)
```

```
a = Shelf([1, 2])
b = Shelf([1, 2])
c = a
a.add(3)
b.add(9)
b.items.append(c.items[:])
c.items.append(7)
```

#Write the output in the space given for the following lines

```
print(a.items)
```

```
print(b.items)
```

```
print(c.items)
```

Coding (45 minutes)

1. Implement the `BrowserHistory` class. A `BrowserHistory` object keeps track of the pages a user has visited in a web browser. (45 minutes)

The main data member is a list called `history`, containing the visited pages in order.

The class should also store an integer called `current`, representing the index of the page that the user is currently viewing.

For example:

```
browser = BrowserHistory("google.com")
```

Initializes:

```
browser.history
# ["google.com"]
browser.current
# 0
```

- `__init__(self, homepage)`: Initialize the browser history with `homepage` as the first page.

```
browser = BrowserHistory("google.com")
print(browser.history)
# ["google.com"]
print(browser.current)
# 0
```

You may assume the homepage is a non-empty string.

- `visit(self, url)`: Visit a new webpage. The new page should be added immediately after the current page, and the browser should now be viewing the new page.

For example:

```
browser = BrowserHistory("google.com")
browser.visit("nyu.edu")
browser.visit("github.com")

print(browser.history)
```

```
# ["google.com", "nyu.edu", "github.com"]
print(browser.current)
# 2
```

There is one important case to consider. Suppose the browser contains:

google.com -> nyu.edu -> github.com -> youtube.com

and the user presses the back button twice. The current page is now "nyu.edu". If the user now visits "wikipedia.org", the pages that were ahead of "nyu.edu" should be removed.

Therefore:

```
browser.visit("wikipedia.org")
```

should result in:

```
["google.com", "nyu.edu", "wikipedia.org"]
```

and not:

```
["google.com", "nyu.edu", "wikipedia.org",
 "github.com", "youtube.com"]
```

In other words, visiting a new page after going backward clears the browser's forward history.

- `back(self)`: Move backward by one page in the browser history. Return the URL of the page that is now being viewed.

Example:

```
browser = BrowserHistory("google.com")
browser.visit("nyu.edu")
browser.visit("github.com")
print(browser.back())
# "nyu.edu"
print(browser.current)
# 1
```

If the browser is already at the first page, it should stay there.

```
browser = BrowserHistory("google.com")
print(browser.back())
# "google.com"
print(browser.current)
# 0
```

- `forward(self)`: Move forward by one page in the browser history. Return the URL of the page that is now being viewed.

Example:

```
browser = BrowserHistory("google.com")
browser.visit("nyu.edu")
browser.visit("github.com")
browser.back()
print(browser.forward())
# "github.com"
```

If the browser is already at the most recently visited page, it should stay there.

- `get_current(self)`: Return the URL of the page currently being viewed.

```
browser = BrowserHistory("google.com")
browser.visit("nyu.edu")
print(browser.get_current())
# "nyu.edu"
```

This method should not modify the object.

- `get_history(self)`: Return a list containing all pages in the browser history. The returned list should be a new list. Therefore, changing the returned list should not change the BrowserHistory object's own history.

Example:

```
browser = BrowserHistory("google.com")
browser.visit("nyu.edu")
pages = browser.get_history()
pages.append("fake.com")
print(browser.history)
# ["google.com", "nyu.edu"]
```

Full implementation example

```
browser = BrowserHistory("google.com")
browser.visit("nyu.edu")
browser.visit("github.com")
browser.visit("youtube.com")
print(browser.get_current())
# youtube.com
print(browser.back())
# github.com
print(browser.back())
```

```
# nyu.edu
print(browser.forward())
# github.com
print(browser.get_history())
# ["google.com", "nyu.edu", "github.com", "youtube.com"]
```

Now consider

```
browser.back()
# current page is nyu.edu
browser.visit("wikipedia.org")
print(browser.get_history())
```

Output:

```
["google.com", "nyu.edu", "wikipedia.org"]
```

The old forward history has been removed.

Starter Template

```
class BrowserHistory:

    def __init__(self, homepage):
        """
        :type homepage: str
        """
        pass

    def visit(self, url):
        """
        :type url: str
        :return type: None
        """
        pass

    def back(self):
        """
        :return type: str
        """
        pass

    def forward(self):
        """
        :return type: str
```

```
    """
    pass

    def get_current(self):
        """
        :return type: str
        """
        pass

    def get_history(self):
        """
        :return type: list
        """
        pass

# OPTIONAL
    def __repr__(self):
        """
        :return type: str
        """
        pass
```

OPTIONAL

Continuation of the above coding question:

- `back(self, steps)` and `forward(self, steps)`: Modify your `back` and `forward` methods so they accept an integer specifying how many pages to move.

For example:

```
browser = BrowserHistory("A")

browser.visit("B")
browser.visit("C")
browser.visit("D")
browser.visit("E")

print(browser.back(3)) # "B"
```

If the step is larger than the number of available pages, move as far as possible.
Assume steps are always a positive number.

```
print(browser.back(100))
# "A"
```

Similarly:

```
print(browser.forward(2))
# "C"
```

- `__repr__(self)`: Return a string showing the browser history, with the current page surrounded by square brackets.

For example:

```
browser = BrowserHistory("google.com")
browser.visit("nyu.edu")
browser.visit("github.com")
browser.back()

print(browser)
```

could produce:

```
google.com -> [nyu.edu] -> github.com
```

2. Implement the `Polynomial` class, where the main data member is a list containing each coefficient from lowest power to highest.

For example, the coefficient list of the polynomial $p(x) = 2x^4 - 9x^3 + 7x + 3$ is `[3, 7, 0, -9, 2]`. Note that 0 is included as $0x^2$:

- `__init__(self, coefficients)`: Initialize the `Polynomial` class with coefficients in reverse order. If no list is passed, `Polynomial` should evaluate to $p(x) = 0$.
- `__add__(self, other)`: Return a **new** `Polynomial` with added coefficients of both `Polynomials` (do not modify the original `Polynomials`).
 - *Example of adding polynomials:*

$$(2x^4 - 9x^3 + x^2 + 7x + 3) + (3x^9 + 9x) = 3x^9 + 2x^4 - 9x^3 + x^2 + 16x + 3$$

- `__call__(self, param)` Return the integer value of the `param` passed in the polynomial equation.
 - For example, if `poly1` represents $2x^2 + x$, `poly1(1)` will return 3 ($2(1)^2 + (1) = 3$)

- `__repr__(self)`: Return a string representation of the polynomial equation. Instead of superscript, we will represent powers using the caret symbol `^`. You may format it as such: $p(x) = 2x^4 - 9x^3 + 7x + 3$
 $2x^4 + -9x^3 + 0x^2 + 7x^1 + 3x^0$
 You can use Python's `join` function, if helpful.
- `__mul__(self, other)`: Return a **new** Polynomial from both polynomials multiplied together.
 - *Example of multiplying polynomials:*
 - $(x+1)*(x+2) = x^2 + 3x + 2$
- `__derive__(self)`: Modify the Polynomial to have its derived value (do not return a new list of values).

Example 1:

```
poly1 = Polynomial([3, 7, 0, -9, 2]); # represents 2x^4 - 9x^3 + 7x + 3
poly2 = Polynomial([2, 0, 0, 5, 0, 0, 3]); # represents 3x^6 + 5x^3 + 2
poly3 = poly1 + poly2
print(poly3.data) # return [5, 7, 0, -4, 2, 0, 3]
print(poly1(1)) # return 3
print(poly2(1)) # return 10
print(poly3(1)) # return 13

# Optional test values
poly1.derive() # returns none
print(poly1) # returns '8x^3 + -27x^2 + 7'
poly4 = poly1 * Polynomial([1,2]);
print(poly4) # return array of 8x3 -27x2 + 7 * (x + 2)
```

Starter Template

```
class Polynomial:
    def __init__(self, coefficients):
        """
        :type coefficients: list
        """
    def __add__(self, other):
        """
```

```
        :type other: Polynomial
    :return type: Polynomial
    """
    def __call__(self, other):
        """
        :type other: Polynomial
        :return type: int
        """
    def __mul__(self, other):
        """
        :type other: Polynomial
        :return type: Polynomial object
        """
    def derive(self):
        """
        :return type: None
        """
    def __repr__(self):
        """
        :return type: str
        """
```